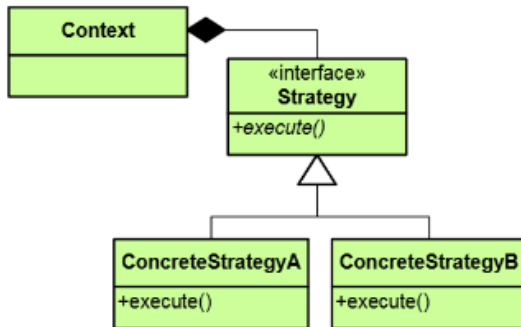


Strategy

Type: Behavioral

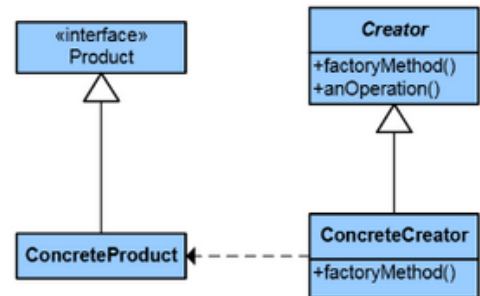
Description: Define a family of algorithms, encapsulate each one, and make them interchangeable. Lets the algorithm vary independently from the clients that use it.



Factory Method

Type: Creational

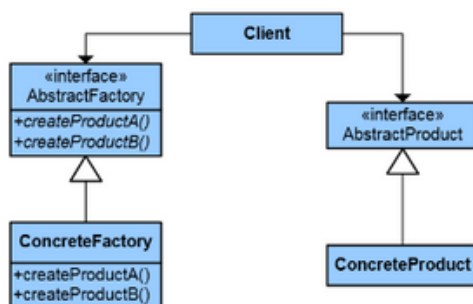
Description: Define an interface for creating an object, but let subclasses decide which class to instantiate. Lets class defer instantiation to subclasses.



Abstract Factory

Type: Creational

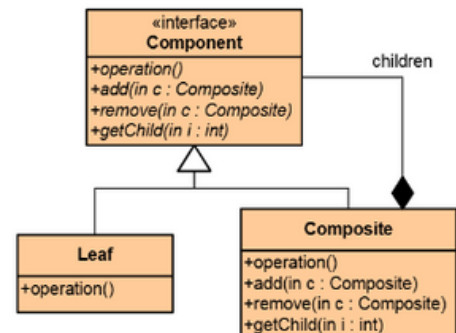
Description: Provides an interface for creating families of related or dependent objects without specifying their concrete class.



Composite

Type: Structural

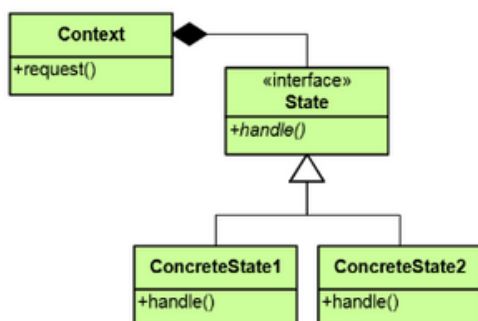
Description: Compose objects into tree structures to represent part-whole hierarchies. Lets clients treat individual objects and compositions of objects uniformly.



State

Type: Behavioural

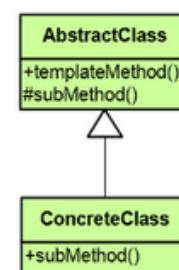
Description: Allow an object to alter its behaviour when its internal state changes. The object will appear to change its class.



Template Method

Type: Behavioural

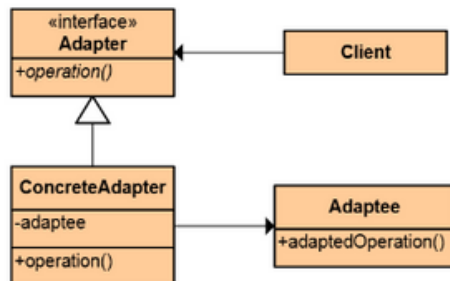
Description: Define the skeleton of an algorithm in an operation, deferring some steps to subclasses. Lets subclasses redefine certain steps of an algorithm without changing the algorithm's structure.



Adapter

Type: Structural

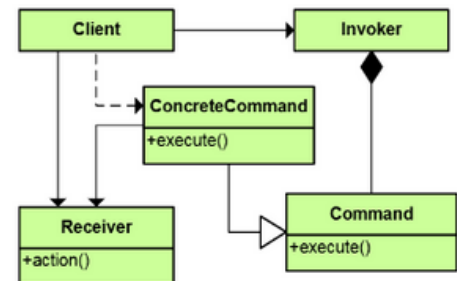
Description: Convert the interface of a class into another interface clients expect. Lets classes work together that couldn't otherwise because of incompatible interfaces.



Command

Type: Behavioural

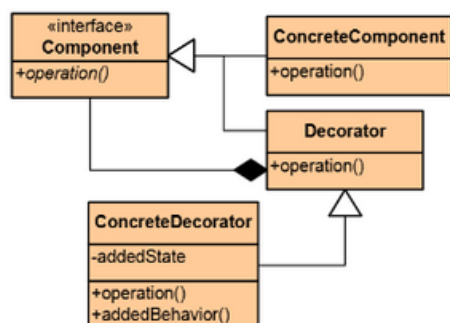
Description: Encapsulate a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations.



Decorator

Type: Structural

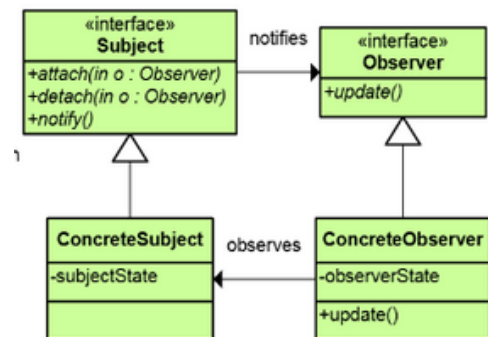
Description: Attach additional responsibilities to an object dynamically. Provide a flexible alternative to sub-classing for extending functionality.



Observer

Type: Behavioural

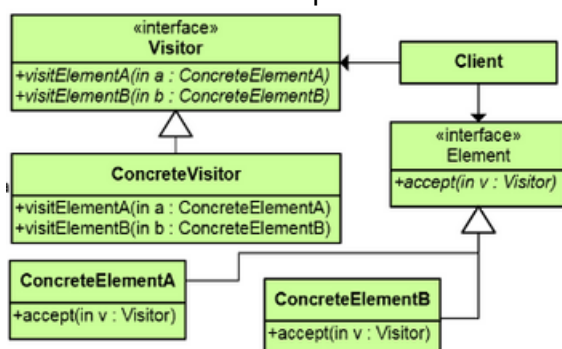
Description: Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.



Visitor

Type: Behavioural

Description: Represent an operation to be performed on the elements of an object structure. Lets you define a new operation without changing the classes of the elements on which it operates.



Prototype

Type: Creational

Description: Specify the kinds of objects to create using a prototypical instance, and create new objects by copying this prototype.

